🧠 **MASTER SYSTEM PROMPT**
ORIOSTER – AI-ASSISTED, OFFLINE-FIRST, MULTI-TENANT HMS
🏗️ 1. SYSTEM ARCHITECTURE OVERVIEW
🔷 Core Principle
Offline-first, cache-first, human-controlled
AI is assistive only, never blocking or authoritative
🔷 Tech Stack (Non-Negotiable)
📱 Frontend & Client Architecture
Framework: Flutter (Compiles to native ARM code for Android, with seamless cross-platform deployment options).
Language: Dart.
State Management: Riverpod 3.0 (Reactive state management for handling complex medical dashboards, forms, and real-time UI updates).
Navigation: GoRouter (Declarative routing for deep-linking and handling strict, role-based authentication guards).
UI Design: Custom Theming / Glassmorphism (Implemented efficiently to maintain 60FPS performance on hospital devices).
🗄️ Local-First & Sync Engine
Sync Engine: PowerSync SDK (Automatically handles background synchronization, network detection, upload queues, and conflict resolution out-of-the-box).
Local Database: SQLite (Embedded relational database that maps perfectly 1:1 with your cloud PostgreSQL schema).
🔒 Security & Encryption
Local Data at Rest: SQLCipher (Encrypts the entire local SQLite database file on the Android device).
End-to-End Encryption (E2EE): Dart Cryptography Library (e.g., cryptography package using AES-GCM to encrypt highly sensitive clinical summaries client-side before they are passed to PowerSync).
Cloud Data Access: Supabase Row Level Security (RLS) (Ensures strict access control at the database level based on user roles like Doctor, Nurse, or Admin).
☁️ Cloud Backend & Source of Truth
Backend-as-a-Service: Supabase.
Cloud Database: PostgreSQL (The highly scalable, relational source of truth).
Authentication: Supabase Auth (Manages JWT tokens and integrates seamlessly with PowerSync for secure connections).
Real-time Publisher: PostgreSQL Logical Replication (Publishes database changes to the PowerSync service so connected devices update instantly).
🧠 AI Integration
AI Gateway: OpenRouter (Multi-model orchestration to route medical summaries, transcriptions, or analysis to the most appropriate and cost-effective LLM without vendor lock-in).
🔷 Architectural Pattern
UI Layer (Flutter + GoRouter)
↓
State Controller (Riverpod)
↓
├──> AI Gateway (OpenRouter)
│ (Triggered via API/Edge Functions for medical summaries & analysis)
↓
Client-Side Encryption (E2EE)
(Encrypts sensitive clinical summaries locally before storage)
↓
Local Database (SQLite + SQLCipher) ← PRIMARY SOURCE OF TRUTH
↓
Sync Engine (PowerSync SDK)
(Handles background queues, offline state, and conflict resolution)
↓
Cloud Backend (Supabase PostgreSQL + Auth + RLS)

👉 Key Rule:
Local system is always authoritative during runtime; cloud sync is eventual.

The step-by-step implementation guide to wire powersync, supabase and flutter technologies together :
Phase 1: Supabase Configuration (The Publisher)
PowerSync needs permission to listen to changes in your Supabase database in real-time. You do this by setting up PostgreSQL logical replication.
* Open the Supabase SQL Editor: Navigate to your Supabase dashboard and open the SQL editor.
* Create a Replication Role: Run the following SQL to create a dedicated user for PowerSync:
CREATE ROLE powersync_role WITH REPLICATION BYPASSRLS LOGIN PASSWORD 'your-secure-password';
GRANT SELECT ON ALL TABLES IN SCHEMA public TO powersync_role;

- Create a Publication: Tell Supabase which tables PowerSync is allowed to sync.

CREATE PUBLICATION powersync FOR TABLE public.patients, public.vitals, public.staff;

Phase 2: PowerSync Cloud Setup (The Bridge)
Next, you will connect the PowerSync service to your Supabase project.

* Create a PowerSync Project: Go to the PowerSync Dashboard and create a new instance.
* Connect the Database: * Go to DB Connections.
  * Paste your Supabase database connection string (URI). Important: You must use the direct connection string (usually db.[your-project-ref].supabase.co on port 5432), not the connection pooler.
  * Update the username and password in the URI to use the powersync_role you created in Phase 1.
* Enable Supabase Auth: In the PowerSync dashboard under the Client Auth tab, enable “Use Supabase Auth”. This ensures PowerSync respects your user tokens.
* Define Sync Rules (YAML): Navigate to Sync Rules. This is where you securely partition data so a nurse only downloads the patients they are authorized to see.

```
Bucket_definitions:
User_patients:
  Parameters:
    - Request.user_id
  Data:
    - SELECT * FROM patients WHERE assigned_nurse_id = request.user_id
```

Phase 3: Flutter Client Integration (The Local App)
Now, bring it into your Flutter codebase.

- Add Dependencies: Add the required packages to your pubspec.yaml:

```
Dependencies:
Supabase_flutter: ^latest
Powersync: ^latest
Sqlite3_flutter_libs: ^latest # Required for the local SQLite engine
```

- Define the Local Schema: You must define your database tables in Dart so PowerSync knows how to structure the local SQLite file.

```
Import ‘package:powersync/powersync.dart’;

Const patientTable = Table(‘patients’, [
  Column.text(‘name’),
  Column.text(‘assigned_nurse_id’),
  Column.text(‘status’),
]);

Final schema = Schema([patientTable]);
```

- Initialize the SDK: When your app starts, initialize both Supabase and PowerSync.

```
Final supabase = Supabase.instance.client;
Final db = PowerSyncDatabase(schema: schema, path: ‘orioster_local.db’);
Await db.initialize();
```

Phase 4: The Sync Connector (The Magic)
To make everything work together, you create a class that extends AbstractPowerSyncBackendConnector. This class acts as the translator between PowerSync and Supabase.
* fetchCredentials(): PowerSync will call this to get the current user’s Supabase JWT token to authenticate the sync connection.
* uploadData(): When the user makes a change offline, PowerSync saves it locally and puts it in an upload queue. When the network returns, PowerSync calls this function. You then use the standard Supabase SDK inside this function to push the queued changes to the cloud (e.g., supabase.from(‘patients’).upsert(data)).

🔄 2. DATA FLOW (STRICT ORDER)
This is the most critical system rule:

User Action
→ Local Validation
→ Local Summarization
→ Encrypted SQLite Cache
→ UI Update (instant)
→ Background Sync (Supabase)
→ AI Call (SUMMARY ONLY)
→ AI Output Cached
🔴 Hard Constraints
❌ No raw PHI leaves device
❌ AI never blocks UI
❌ Network never required
✅ Design Implication
App must feel instant even offline
AI is async + non-blocking
All failures must degrade gracefully

🧠 3. AI SYSTEM (ORIO AI)
🔷 Role
Clinical decision support only
Never gives final diagnosis
Must include disclaimer always
🔷 4-Tier Model Routing
Each function uses failover chain:
Tier
Purpose
Tier 1
Best quality
Tier 2
Fast fallback
Tier 3
Reasoning / efficiency
Tier 4
Reliable minimum

```
# ORIO AI: CONFIGURATION & CREDENTIALS
# ========================================
# ⚠️ REPLACE WITH YOUR ACTUAL KEY BEFORE RUNNING
OPENROUTER_API_KEY = "sk-or-YOUR_ACTUAL_API_KEY_HERE"
APP_NAME = "Orioster"
# ========================================
# THE 4-TIER MODEL MATRIX (STRICTLY FREE)
# ========================================
# Tiers 1-2: High-Performance Enterprise Models (via Free Endpoints)
# Tiers 3-4: Open Source Efficiency Models (Always Free)
MODEL_TIERS = {
# 1. Monitoring Smart Prescription (Rigid Logic)
"rx_monitoring": [
"qwen/qwen-2.5-72b-instruct:free",        # Tier 1: Logic Monster
"meta-llama/llama-3.3-70b-instruct:free", # Tier 2: Instruction Follower
"deepseek/deepseek-r1:free",              # Tier 3: Reasoning Engine
"mistralai/mistral-nemo:free"             # Tier 4: Reliable Fallback
],
# 2. Doctor Prescription & PDF Gen (Formatting)
"rx_generation": [
"meta-llama/llama-3.3-70b-instruct:free", # Tier 1: Best Formatting
"google/gemini-2.0-flash-exp:free",       # Tier 2: High Speed
"qwen/qwen-2.5-72b-instruct:free",        # Tier 3: Structured Data
"google/gemma-2-9b-it:free"               # Tier 4: Efficiency
],

# 3. Billing & Invoice Gen (Speed & Math)
"billing": [
"google/gemini-2.0-flash-exp:free",       # Tier 1: Flash Speed
"google/gemma-2-9b-it:free",              # Tier 2: Fast Arithmetic
"mistralai/mistral-nemo:free",            # Tier 3: Stable
"microsoft/phi-3-medium-128k-instruct:free" # Tier 4: Logic Optimized
],

# 4. Appointment Scheduling (Conversational)
"scheduling": [
"meta-llama/llama-3.3-70b-instruct:free", # Tier 1: Human-like Tone
"google/gemini-2.0-flash-exp:free",       # Tier 2: Time Logic
"deepseek/deepseek-chat:free",            # Tier 3: Chat Optimized
"mistralai/mistral-small-24b-instruct:free" # Tier 4: Instruction Following
],

# 5. Treatment Suggestion (Knowledge Retrieval)
"treatment": [
"google/gemini-2.0-flash-exp:free",       # Tier 1: Latest Knowledge Base
"deepseek/deepseek-chat:free",            # Tier 2: Strong MedQA Scores
"meta-llama/llama-3.3-70b-instruct:free", # Tier 3: General Knowledge
"qwen/qwen-2.5-72b-instruct:free"         # Tier 4: Logical Backup
],

# 6. Differential Diagnosis (Complex Reasoning)
```

“diagnosis”: [
“deepseek/deepseek-r1:free”,   # Tier 1: “Reasoning” Model (Thinks first)
“google/gemini-2.0-flash-thinking-exp:free”, # Tier 2: Google Thinking Model
“qwen/qwen-2.5-72b-instruct:free”,   # Tier 3: Lateral Thinking
“meta-llama/llama-3.3-70b-instruct:free” # Tier 4: Standard Opinion
].

7. Lab Report Analysis (Context Window)
“lab_analysis”: [
“google/gemini-2.0-flash-exp:free”,   # Tier 1: 1M Token Context
“deepseek/deepseek-chat:free”,   # Tier 2: 64k Context
“microsoft/phi-3-medium-128k-instruct:free”, # Tier 3: 128k Context
“meta-llama/llama-3.3-70b-instruct:free” # Tier 4: Summarizer
].

8. Imaging Analysis (Vision/Multimodal)
“imaging”: [
“google/gemini-2.0-flash-exp:free”,   # Tier 1: Native Multimodal
“qwen/qwen-2.5-vl-72b-instruct:free”,   # Tier 2: Best Open Vision
“meta-llama/llama-3.2-90b-vision-instruct:free”, # Tier 3: Llama Vision
“meta-llama/llama-3.2-11b-vision-instruct:free” # Tier 4: Fast Vision
]

}
# ==
# CORE ENGINE: INTELLIGENT ROUTING
# ==
Def encode_image(image_path):
“” Helper to convert images to Base64 for the API.”””
If not image_path or not os.path.exists(image_path):
Return None
With open(image_path, “rb”) as image_file:
Return base64.b64encode(image_file.read()).decode(‘utf-8’)
Def run_orio_task(function_name, user_input, system_prompt, image_path=None):
“””

Executes the task using the 4-Tier Strategy.
Automatically fails over if a model is busy or errors out.
“””
# 1. Get the model chain
Model_chain = MODEL_TIERS.get(function_name)
If not model_chain:
Return “❌ Error: Invalid Function Name.”
Print(f”\n🚀 ORIO AI: Starting Task ‘{function_name}’”)
# 2. Handle Image Encoding (if needed)
Base64_image = None
If function_name == “imaging” and image_path:
Base64_image = encode_image(image_path)
If not base64_image:
Return “❌ Error: Image file not found.”
3. Iterate through Tiers (1 -> 4)
For tier, model_id in enumerate(model_chain, 1):
Print(f”   [Tier {tier}] Attempting: {model_id}…”)
# Construct Messages
Messages = []
If base64_image:
# Multimodal Payload
Messages = [{
“role”: “user”,
“content”: [
{“type”: “text”, “text”: system_prompt + “\n\n” + user_input},
{“type”: “image_url”, “image_url”: {“url”: f”{base64_image}”}}
]
}]
Else:
# Standard Text Payload
Messages = [
{“role”: “system”, “content”: system_prompt},
{“role”: “user”, “content”: user_input}

]

Payload = {
“model”: model_id,
“messages”: messages,
“temperature”: 0.2, # Low temperature for medical precision
“top_p”: 0.9
}
Try:
# API Call
Response = requests.post(
https://openrouter.ai/api/v1/chat/completions,
headers={
“Authorization”: f”Bearer {OPENROUTER_API_KEY}”,
“HTTP-Referer”: SITE_URL,
“X-Title”: APP_NAME,
“Content-Type”: “application/json”
},
Data=json.dumps(payload),
Timeout=15 # Fail fast (15s) to switch to next tier if busy
)
Check Success
If response.status_code == 200:
Result = response.json()
If ‘choices’ in result and len(result[‘choices’]) > 0:
Content = result[‘choices’][0][‘message’][‘content’]
Print(f” ✅ Success with Tier {tier}!”
Return content
# If we get here, the API returned an error (e.g., 429 Busy)
Print(f” ⚠️ Tier {tier} Failed (Status {response.status_code}). Switching to next tier…”)
Except Exception as e:
Print(f” ❌ Tier {tier} Error: {e}. Switching to next tier…”)
# Small delay before retry to be polite to the API
Time.sleep(1)
Return “🚨 CRITICAL FAILURE: All 4 Tiers are unresponsive. Please check internet or API Key.”

🔷 Execution Behavior
Timeout: ≤ 15s
Auto failover
No blocking retries
🔷 Output Format (Strict JSON)
JSON
{
"summary": "...",
"risk_level": "low | moderate | high",
"confidence": 0.0–1.0,
"limitations": ["..."],
"recommendation_type": "advisory"
}
Mandatory disclaimer:
"This output is not a diagnosis and must be reviewed by a human professional."

🤌 4. AI SAFETY RULES
❌ AI NEVER RECEIVES
Names
Phone numbers
IDs
Addresses
Raw reports/images
Free text notes
✅ AI ONLY RECEIVES
patient_summary_v1
De-identified
Token-compressed (≥70% reduction)
🤌 Security Strategy
Local summarization acts as privacy firewall
AI logs are:
Encrypted

Scoped per user
PHI-safe

🔔 5. OFFLINE-FIRST BEHAVIOR
Core Guarantees
Full functionality offline
Autosave everywhere
Local attachments first
AI Behavior
Condition
Behavior
Offline
AI disabled
Online later
Queue executes
Failure
Retry with backoff

📋 6. PATIENT ENTRY WORKFLOW (CRITICAL MODULE)
🔴 Entry Point (Non-Negotiable)

+ Add Patient (FAB)
→ Always visible
→ One tap → Entry flow
🔟 UPDATED PATIENT ENTRY FLOW (DEVELOPER SPEC)

1. Patient's General Informations

🎯 Purpose
Capture core identity and demographic baseline for the patient.
✅ Validation Rules
Required fields: Name, age, gender, address, contact number, local ID, optional info – blood group, height, weight, profession, highest level of education.
No duplicates (local ID check)
Consent flag must be present (if applicable)
🔔 Offline Behavior
Instantly saved to Powersync (draft state)
Generates temporary patient ID
😈 AI Constraint
❌ No AI access
Raw PHI strictly local

2. Chief Complaint

🎯 Purpose
Define the primary reason for visit (core clinical anchor).
✅ Validation Rules
Must not be empty
Prefer structured input (dropdown + optional text)
Limit free-text length
🔔 Offline Behavior
Autosaved locally
Indexed for summary generation later
😈 AI Constraint
❌ No AI usage
Will later be compressed into summary

3. Past History

🎯 Purpose
Record prior diseases, surgeries, chronic conditions.
✅ Validation Rules
Structured tags (e.g., diabetes, hypertension)
Optional notes allowed but capped
🔔 Offline Behavior
Stored in structured format for summary engine
😈 AI Constraint
❌ No AI
Must be pre-structured for token reduction

4. Ongoing Medications

🎯 Purpose
Capture current drug usage (critical for safety).

✅ Validation Rules
Drug name required
Dose/frequency optional but recommended
Duplicate drug warning
🔕 Offline Behavior
Stored locally
Linked to future Rx monitoring AI (post-submit only)
🤖 AI Constraint
❌ No AI at this stage
Data prepared for safe summary extraction

5. Vitals

🎯 Purpose
Capture measurable clinical indicators.
✅ Validation Rules
Numeric validation (BP, HR, Temp, etc.)
Range checks (flag abnormal values)
🔕 Offline Behavior
Stored instantly
Can trigger local triage indicators (no AI needed)
🤖 AI Constraint
❌ No AI
Local logic handles alerts

6. Hyper-sensitivity

🎯 Purpose
Record allergies / adverse reactions.
✅ Validation Rules
At least “None” or one entry required
Severity tagging (mild/moderate/severe)
🔕 Offline Behavior
Stored locally
Used later for clinical safety checks
🤖 AI Constraint
❌ No AI
Must remain structured

7. Assign Doctor (Appointment Creation)

🎯 Purpose
Link patient to a doctor via appointment system.
✅ Validation Rules
Doctor selection required
Time slot must exist
Status = Scheduled (default)
🔕 Offline Behavior
Appointment created locally
Syncs later with Supabase
🤖 AI Constraint
❌ No AI
Pure scheduling logic

8. Local Summary (CRITICAL STEP)

🎯 Purpose
Generate patient_summary_v1 — the AI-safe dataset
✅ Validation Rules
Must include:
Chief complaint
Key history
Medications
Vitals
Minimum compression target: ≥70%
🔕 Offline Behavior
Fully local computation
Cached in Isar
🤖 AI Constraint 🚨
❌ STRICTLY NO AI
❌ NO NETWORK
✅ This is the privacy firewall

9. Notify Assigned Doctor (AI-Assisted Summary Delivery)

🎯 Purpose

Send a clean, summarized patient condition to the assigned doctor.
✅ Validation Rules
Requires:
Completed summary (Step 8)
Assigned doctor (Step 7)
Notification must be queued if offline
🔕 Offline Behavior
If offline:
Queue notification locally
If online:
Send via push/system notification
😈 AI Constraint
✅ AI MAY be used here, BUT:
Input = patient_summary_v1 ONLY
Output = enhanced readable summary (optional)
👉 Example use:
Tone adjustment
Prioritization (urgent vs routine)
❌ NEVER include:
Raw PHI
Attachments

10. Review & Submit

🎯 Purpose
Finalize and lock patient record.
✅ Validation Rules
All steps completed
Role-based checklist passed
Summary exists
🔕 Offline Behavior
Final state saved to Isar
Sync queued to Supabase
😈 AI Constraint
❌ No automatic AI trigger
AI is user-initiated only after submission
🔐 SYSTEM-LEVEL ENFORCEMENT
🚫 Hard Stops
No skipping steps
No AI before Step 8
No submission without checklist
🔁 Data Lifecycle
Plain text
Draft → Structured Data → Local Summary → Cached → Synced → AI (optional)
🧠 KEY ARCHITECTURAL INSIGHT
🔥 This New Flow Introduces a Critical Shift:
OLD:
AI after submission
NEW:
AI-assisted doctor notification BEFORE final submit
👉 This enables:
Faster triage
Real-time awareness
Better coordination in camps/emergency setups
⚠️ DESIGN RISKS (IMPORTANT)
You should handle carefully:

1. Notification Timing

Avoid duplicate sends (use idempotent queue)

2. AI Misuse Risk

Enforce strict input filter → summary only

3. Offline Queue Complexity

Notification queue must persist across restarts
✅ FINAL TAKEAWAY
This workflow is now:
A real-time, offline-capable patient intake + doctor alert system with AI-safe boundaries
The most critical step remains Step 8 (Local Summary) —
Everything AI-related depends on it being correct, compressed, and clean.

🧩 7. STEP 9 — LOCAL SUMMARY (CRITICAL)

Purpose
Generate patient_summary_v1
Rules
❌ No network
❌ No AI
✅ Pure local computation
✅ Structured + compressed
👉 This is the AI input boundary layer

✅ 8. ROLE-BASED CHECKLIST SYSTEM
Example
JSON
```
{
  "doctor": ["diagnosis", "treatment_plan"],
  "nurse": ["vitals", "triage_level"]
}
```
Behavior
Blocks submission
Stored locally
Logged for audit

📊 9. PATIENT REPORTS SYSTEM
Widget: Patient Reports Overview
Features
Patient list
Status: Draft / Synced / Reviewed
AI report indicator
Export options
Data Priority

Isar → Supabase (background)
Rules
❌ Never triggers AI automatically
❌ Never waits for network

🔁 10. SYNC & STORAGE STRATEGY
Lifecycle

Draft
→ Summarized
→ Cached (Isar)
→ Synced (Supabase)
→ Verified
Conflict Handling
Server wins
Local archived
Encryption
Mandatory (AES-256)

📦 11. UI + STATE STRUCTURE
Core Controller (Riverpod)
PatientEntryController (StateNotifier / AsyncNotifier)
```
├── State Variables
│   ├── currentStep (int: 1-10)
│   ├── draftPatientId (String? – local temporary ID)
│   ├── role (Enum: Doctor, Nurse, Admin)
│   ├── checklistState (Map<String, bool> - role-based requirements)
│   ├── syncStatus (Enum: Draft, Queued, Synced, Conflict)
│   ├── patientData (Object/Map: Holds steps 1-7 raw data securely in memory)
│   ├── localSummary (String? – patient_summary_v1 generated at Step 8)
│   └── notificationStatus (Enum: Pending, Sent, Failed – for Step 9)
│
└── Core Methods
    ├── updateField(step, data) // Updates local memory state
    ├── saveDraftLocally() // Persists current state to PowerSync SQLite
    ├── generatePrivacyFirewallSummary() // Executes Step 8 (Local, No AI)
    ├── triggerAIAssistedNotification() // Executes Step 9 (Uses OpenRouter)
    ├── validateChecklist() // Enforces Step 10 role limits
```

└── submitFinalRecord() // Locks record and queues for Supabase sync

🎯 12. KEY DESIGN PRINCIPLES (TAKEAWAYS)
⚡ Performance
Instant UI updates
No loading blockers
🔒 Privacy
Local-first
AI sandboxed via summaries
🧠 AI Philosophy
Assistive, not authoritative
Always transparent
📱 Reliability
Works fully offline
Sync is invisible to user
🚀 FINAL DEVELOPER INSIGHT
Orioster is essentially:
A local-first clinical operating system with asynchronous AI augmentation
If you implement only one thing correctly, it must be:
👉 The Data Flow + Local Summary Boundary
That's where:
Privacy is enforced
AI is controlled
System reliability is guaranteed
Choose a number to continue chat:
Design the Flutter UI code structure for the Patient Entry Flow (Step-by-step widgets)
Generate the Isar database schema for all 16+ tables with relationships
Build the AI service layer (OpenRouter integration with failover logic in production-ready code)

Ai functions :

- Extraction & formatting json

- Orio Ai (Doctor's Assistant) : A dedicated Module named Orio Ai that helps the doctor by generating 3 most probable diagnosis with strength metre based on the patient's information, history & signs, symptoms. Once 1 of the 3 probable diagnosis is selected as confirmatory diagnosis by the doctor, then ai must suggest the treatment plan with specific advice to patient for that confirmatory diagnosis (keeping in mind the patient's age, co-morbidity), mention the possible complications and drug interactions (so that doctor may adjust the treatment plan accordingly)

- Generate Prescription : the ai must generate a prescription with all info of the patient, chief complaints, sign & symptoms, confirmatory diagnosis selected by doctor, treatment plan finalized by doctor, advice finalized by doctor. On completion of the above steps, the doctor press "Generate Prescription" button. The ai confirms the availability of all the above info, then generates a customizable prescription. Then the doctor can either export, print or customize the prescription. This function will be inside Orio Ai module.

- Generate & Analyze Medical Investigation Reports : In the Reports module, through a fixed template, all types of medical investigations report will be in the app. When the lab technologist select a specific investigation report, ai must ask for the values of each parameters of that specific report, then generate the report. After that Ai must analyze the report whether the report is normal or not and give a feedback advice accordingly.

- Generate Invoice : In the Invoice module, through a fixed template, the Ai must generate invoice. The Ai must keep the tract of billing for each patient individually.

- AI Hub : In this module, the user can generate Invoice, Medical Investigation Reports, Doctors Prescriptions and Medical Certificates through fixed template for each, the Ai must generate these according to the user's command.

Architecture :